

Q&A Preparation — Deep Learning Project

Video Object Tracking on MOT16 Using Mask R-CNN and Siamese Re-Identification

1. Project Overview Questions

Q: What is the goal of this project?

A: Build an end-to-end multi-object tracking (MOT) pipeline for pedestrians on the MOT16 benchmark. The system detects, re-identifies, and tracks multiple people across video frames with persistent IDs.

Q: What is the pipeline architecture?

A: Three-stage offline tracking:

Frame → **Detect** (Mask R-CNN) → **Crop** → **ReID Embed** (Siamese) → **Hungarian Match** → **Track**

1. **Detection** — Fine-tuned Mask R-CNN (ResNet-50-FPN backbone) detects person bounding boxes
2. **Re-Identification** — Improved Siamese network extracts 256-dim appearance embeddings from person crops
3. **Tracking** — DeepSORT-style tracker uses Hungarian algorithm with combined IoU + cosine distance cost matrix

Q: What datasets did you use?

A:

- **MOT16** — 7 training + 7 test sequences of pedestrian video. Used for detector fine-tuning and tracking evaluation. Ground truth format: (frame, id, bb_left, bb_top, bb_width, bb_height, conf, class, visibility)
- **Market-1501** — 32,668 images of 1,501 identities from 6 cameras. Split: 12,936 train (751 IDs), 3,368 query, 19,732 gallery. Used for ReID training.

Q: Why use two different datasets?

A: MOT16 has limited training data (only 7 sequences) — not enough identities for learning discriminative person embeddings. Market-1501 is a large-scale ReID benchmark specifically designed for learning appearance features across camera views, providing 751 distinct identities for training.

2. Detection — Mask R-CNN

Q: Why Mask R-CNN instead of Faster R-CNN (as suggested by the assignment)?

A: Mask R-CNN extends Faster R-CNN with an additional mask prediction head, providing **instance segmentation masks** alongside bounding boxes. This gives us:

- Richer visual output (mask overlays in tracking videos)
- Potential for mask-based features in future work
- The box predictor is identical to Faster R-CNN's, so detection quality is comparable

Q: How did you adapt the pretrained model?

A: Started from **COCO-pretrained** weights (91 classes), then replaced the classification head with a `FastRCNNPredictor(in_features, num_classes=2)` for binary detection (background + person). This is **transfer learning** — the backbone features from COCO transfer well to pedestrian detection.

```
model = torchvision.models.detection.maskrcnn_resnet50_fpn(weights='DEFAULT', min_s:
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes=2)
```

Q: What were the training hyperparameters?

Parameter	Value
Optimizer	AdamW (Heads: 3e-3, Backbone: 3e-4), weight decay=1e-4
LR schedule	Linear warmup (4 epochs) + Cosine annealing
Epochs	40 (with early stopping, patience=6)

Batch size	8
Data split	80/20 train/val (4253/1063 images)
Input resolution	min_size=384, max_size=640
Gradient clipping	max norm = 1.0
GPU	NVIDIA A100 (AMP / mixed precision)

Q: Why use differential learning rates?

A: This is a fine-tuning phase, not training from scratch. The backbone ($3e-4$) gets a lower LR because it already has excellent features from COCO pretraining — a large LR would destroy those learned features. The heads ($3e-3$) get a $10\times$ higher LR because they were randomly initialized and need to adapt quickly to our 2-class task.

Q: What was the training progression?

Phase	Train Loss	Val Loss
Warmup completed (Epoch 4)	~0.58	~0.55
Mid-training (Epoch 20)	~0.42	~0.44
Best model (Epoch ~35)	~0.36	0.3783
Early stop (Epoch 41)	~0.35	~0.39

Q: Why reduce the input resolution from default (800, 1333)?

A: GPU memory constraints on Google Colab. Mask R-CNN with segmentation masks is memory-intensive. Reducing to (384, 640) allowed training within Colab's A100 memory limits while still achieving good detection quality.

Q: What detection threshold did you use?

A: 0.5 at inference. Only detections with confidence ≥ 0.5 are retained, filtering out low-confidence false positives.

3. Re-Identification — Improved Siamese Network

Q: Describe the network architecture.

A:

```
ResNet-50 backbone → Global Avg Pool → FC(2048→512) → BN → ReLU → Dropout(0.2) → FC(512→256)
→ BN Neck (BatchNorm1d(256))
→ Classifier (Linear(256→751, no bias)) [training only]
```

- Output: L2-normalized 256-dimensional embedding vectors
- During inference, only the normalized embeddings are returned (classifier head is dropped)

Q: What is the BN Neck and why is it important?

A: The **Batch Normalization Neck** (from Luo et al., 2019 "Bag of Tricks for ReID") is a `BatchNorm1d` layer placed between the embedding and the classifier. It **separates the two loss objectives**:

- **Triplet loss** operates on raw (pre-BN) L2-normalized embeddings — optimizes metric space
- **Cross-entropy loss** operates on BN-transformed features — provides classification supervision
- This separation prevents the two losses from fighting each other, improving both discriminative power and generalization

Q: What is the loss function?

A: Combined dual loss:

$$L = L_{\text{triplet}} + 0.5 \times L_{\text{CE}}$$

- **Triplet Loss** (margin $\alpha = 0.3$): On pre-BN embeddings. Pulls same-identity embeddings closer, pushes different-identity embeddings apart.
- **Label-Smoothed Cross-Entropy** ($\epsilon = 0.1$): On post-BN features. Provides identity classification supervision. Label smoothing prevents overconfident predictions and improves generalization.

Q: What is triplet loss? How does it work?

A: For each triplet (anchor, positive, negative):

$$L = \max(0, ||f(\text{anchor}) - f(\text{positive})||^2 - ||f(\text{anchor}) - f(\text{negative})||^2 + \alpha)$$

- Minimizes distance between anchor and positive (same person)
- Maximizes distance between anchor and negative (different person)
- $\alpha = 0.3$ is the margin that enforces a minimum gap

Q: What is batch hard negative mining?

A: Instead of using random negatives, we find the **hardest negative** in each batch — the negative sample closest to the anchor in embedding space. This provides the most informative gradient signal and accelerates learning.

```
dist_mat = torch.cdist(ea, en, p=2)
mask = la.unsqueeze(1) == ln.unsqueeze(0)
dist_mat.masked_fill_(mask, float('inf'))
hard_en = en[torch.argmax(dist_mat, dim=1)]
```

Q: What is label smoothing and why use it?

A: Instead of hard labels (one-hot), we use soft labels:

```
target = (1 - ε) × one_hot + ε / num_classes
```

With $\epsilon = 0.1$ and 751 classes, this prevents the model from becoming overconfident about training identities, improving **generalization** to unseen identities (which is exactly what happens during tracking).

Q: What learning rate schedule did you use?

A: **Linear warmup (10 epochs) + Cosine annealing (70 epochs)**

- Warmup: LR increases linearly from 0 to $3.5e-4$ over 10 epochs. This stabilizes early training when gradients are noisy.
- Cosine annealing: LR smoothly decays following a cosine curve, reaching near 0 by epoch 80. Avoids sudden drops that could destabilize training.

Q: What data augmentations were used?

A:

- **Random horizontal flip** ($p=0.5$) — simulates left/right views
- **Pad(10) + Random crop** — simulates translation/slight viewpoint change

- **Color jitter** (brightness=0.2, contrast=0.15, saturation=0.1, hue=0.05) — handles lighting variations
- **Random erasing** (p=0.5) — simulates occlusion by replacing random patches with noise

Q: What were the evaluation results?

Metric	Value
mAP	73.93%
Rank-1	89.22%
Rank-5	96.05%
Rank-10	97.45%

- **mAP** = Mean Average Precision across all queries. Measures overall retrieval quality.
- **Rank-1** = Probability that the top-1 retrieved gallery image is correct (89.22% means 89% of queries find the right person as #1 result)

Q: How was evaluation conducted?

A: Standard Market-1501 protocol:

- Query each of the 3,368 query images against the 19,732 gallery images
- Exclude same-camera, same-ID images (junk filtering)
- Exclude distractor IDs (-1 and 0000)
- Compute CMC curve and mAP

Q: How does your model compare to the baseline?

A: The baseline Siamese network (from the original assignment) achieved **Rank-1: 54.42%**. Our improved version achieved **Rank-1: 89.22%** — a **35 percentage point improvement**. Key improvements:

1. ResNet-50 backbone (vs. shallow CNN)
2. BN Neck architecture
3. Dual loss (triplet + label-smoothed CE)
4. Data augmentation (especially random erasing)
5. LR warmup + cosine schedule
6. Larger 256-dim embedding
7. Batch hard negative mining

4. Tracking — DeepSORT-Style Tracker

Q: How does the tracker work?

A: Frame-by-frame association:

1. Run detector → get bounding boxes
2. Crop detections, extract 256-dim ReID embeddings
3. Build cost matrix: $C[i,j] = 0.5 \times (1 - \text{IoU}) + 0.5 \times \text{cosine_distance}$
4. Solve assignment with **Hungarian algorithm** (`linear_sum_assignment`)
5. Accept matches with $\text{cost} \leq 0.7$
6. Update matched tracks (box + EMA embedding update)
7. Create new tracks for unmatched detections; increment missed counter for unmatched tracks
8. Delete tracks missing for > 90 frames

Q: What is the Hungarian algorithm?

A: Also known as the **Kuhn-Munkres algorithm**. It finds the optimal one-to-one assignment between tracks and detections that minimizes total cost. It runs in $O(n^3)$ time. We use `scipy.optimize.linear_sum_assignment`.

Q: Why combine IoU and cosine distance?

A: Each has complementary strengths:

- **IoU** — good for short-term matching (nearby frames), spatial proximity
- **Cosine distance** — good for appearance matching, handles occlusions and reappearances
- 50/50 weighting balances spatial and appearance cues

Q: What is the exponential moving average (EMA) embedding update?

A: $e' = 0.8 \times e_{\text{old}} + 0.2 \times e_{\text{new}}$ This provides robustness against transient appearance changes (e.g., partial occlusion, lighting changes) while slowly adapting the track's appearance model. The 0.8/0.2 weighting means the track "remembers" most of its history but gradually incorporates new observations.

Q: Does the tracker have a gallery/re-identification feature?

A: Yes! When a track is deleted (missed $> \text{max_missed frames}$), its ID and embedding are saved to a **gallery**. When a new unmatched detection appears, the tracker first checks the gallery for a match (cosine distance < 0.7). This allows **recovering the same ID** when a person reappears after occlusion.

Q: What were the tracking results?

Metric	Value
Sequence	MOT16-09
Total frames	525
Unique IDs	22
Total track entries	4,480
Active tracks/frame	~8–12

Q: Why didn't you compute MOTA/IDF1/HOTA?

A: Time constraints. These standard MOT metrics require careful ground truth alignment and specialized evaluation toolkits (e.g., py-motmetrics). We relied on qualitative video evaluation and track count statistics instead.

5. Challenges & Solutions

Q: How did you handle limited training data?

A: Transfer learning — COCO-pretrained backbone, only fine-tuning the detection head. Plus data augmentation (flip, crop, color jitter).

Q: How did you handle GPU memory constraints?

A: Reduced input resolution (384×640 vs default 800×1333), checkpoint saving for cross-session resume, early stopping to avoid wasted compute.

Q: How did you handle the architecture mismatch between detector and ReID?

A: The detector (2-class) and ReID model (751-class) have different checkpoint structures. For the detector, we use **filtered checkpoint loading** that skips mask predictor keys with shape

mismatches:

```
filtered = {k: v for k, v in state.items() if 'mask_predictor' not in k}
model.load_state_dict(filtered, strict=False)
```

Q: How did you handle identity switches in crowded scenes?

A: Combined IoU + cosine distance cost function (equal 0.5 weight). The EMA embedding update provides temporal smoothing. The gallery-based re-identification recovers IDs after occlusion.

6. Deeper Conceptual Questions

Q: What is Feature Pyramid Network (FPN) and why is it used?

A: FPN builds a multi-scale feature pyramid from the backbone. It combines high-resolution, semantically weak features (early layers) with low-resolution, semantically strong features (deep layers) via top-down pathway and lateral connections. This helps detect objects at different scales — critical for MOT16 where pedestrians vary greatly in size.

Q: What is ResNet-50 and why use it?

A: A 50-layer deep residual network with skip connections. Skip connections solve the vanishing gradient problem, allowing much deeper networks to be trained. It's the de facto standard backbone for detection and ReID tasks.

Q: What's the difference between online and offline tracking?

A:

- **Online:** Processes frames sequentially (causal), can't use future information. Used for real-time applications.
- **Offline:** Can process batches of frames, potentially using future information. Better recovery from occlusions.
- Our system is **offline** in the sense that we process pre-recorded sequences, but the tracker itself operates **frame-by-frame** (online-style).

Q: Why not use a Kalman filter (as in original DeepSORT)?

A: We simplified the tracker by omitting the Kalman filter motion model. This was a **deliberate tradeoff** — the Kalman filter predicts where a track will be in the next frame, which helps during occlusions and fast motion. Without it, we rely more heavily on appearance features. This is listed as a limitation.

Q: What is the difference between cosine distance and Euclidean distance?

A:

- **Cosine distance** = $1 - \cos(a, b)$. Measures angular difference, invariant to vector magnitude. Good for comparing normalized embeddings.
- **Euclidean distance** = $\|a - b\|_2$. Measures absolute spatial distance. Since we L2-normalize our embeddings, Euclidean and cosine distance are monotonically related.

Q: What is the ReID domain gap problem?

A: Market-1501 (training) and MOT16 (testing) have different characteristics:

- Different camera viewpoints and resolutions
- Different lighting conditions
- Different pedestrian appearances and clothing This means ReID features learned on Market-1501 may not generalize perfectly to MOT16. This is a known limitation.

Q: How would you improve the system?

A:

1. Add a **Kalman filter** for motion prediction
2. Use **domain adaptation** to bridge Market-1501 → MOT16 gap
3. Compute formal **MOTA/IDF1** metrics
4. Test on more challenging sequences (e.g., MOT16-03 with high density)
5. Use **camera-compensated motion** models
6. Implement **cascade matching** (confirmed vs. tentative tracks)
7. Fine-tune ReID model on MOT16 data itself

Q: What grade do you think you deserve?

A: A/B. The system:

-  Successfully detects pedestrians across all frames
-  Assigns bounding boxes with persistent identity labels

-  Tracks objects across the majority of frames
-  Produces annotated video output
- Some ID switches occur in challenging scenarios (limitation)

7. Quick Reference — Key Numbers

Component	Key Metric	Value
Detector	Best val loss	0.3783
Detector	Input size	384×640
Detector	Threshold	0.5
Detector	Training epochs	40 (early stop ~35)
Detector	Optimizer	AdamW (heads 3e-3, backbone 3e-4)
ReID	mAP	73.93%
ReID	Rank-1	89.22%
ReID	Embedding dim	256
ReID	Training epochs	80 (best at 76)
ReID	Best total loss	1.5971
Tracker	IoU weight	0.5
Tracker	Max cost	0.7
Tracker	Max missed	90 frames
Tracker	EMA factor	0.8 old + 0.2 new
Tracker	Unique IDs (MOT16-09)	22
Tracker	Total frames	525